What is Docker and why we should use it?

MLSecOps in Action Course

Mohammad Fozouni

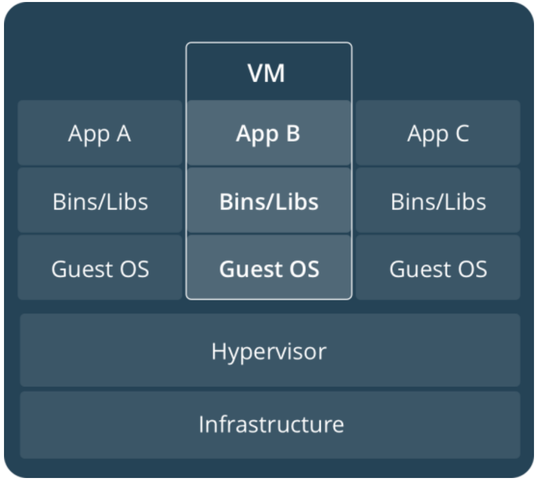
What is Docker?

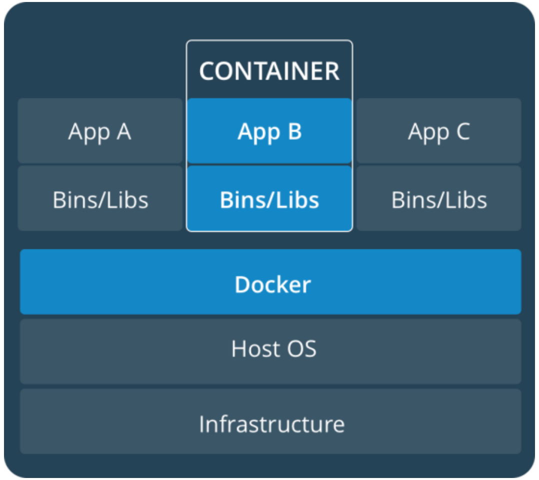
Docker is a software platform that allows you to **build**, **test**, and **deploy** applications quickly, **packaging** software into standardized units called **containers.**

Why Docker is helpful?

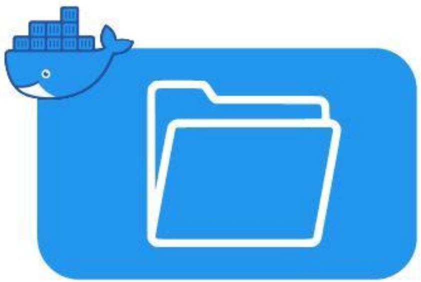
Regarding containers

- Container ≠ VM
- Isolated
- Share OS
- and sometimes bins/libs



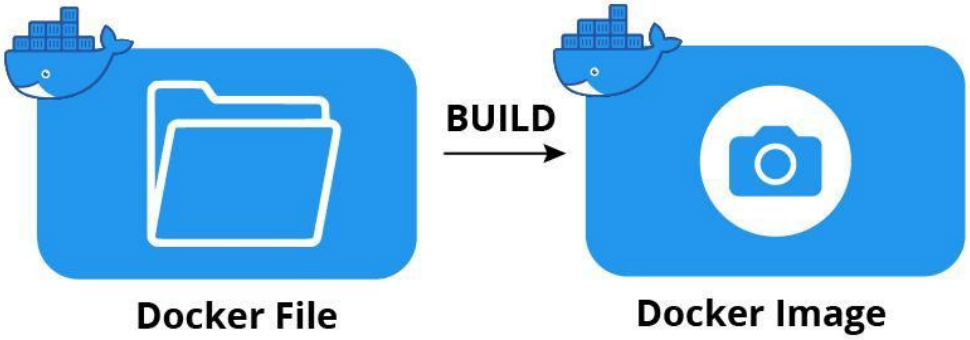


What is the building process?

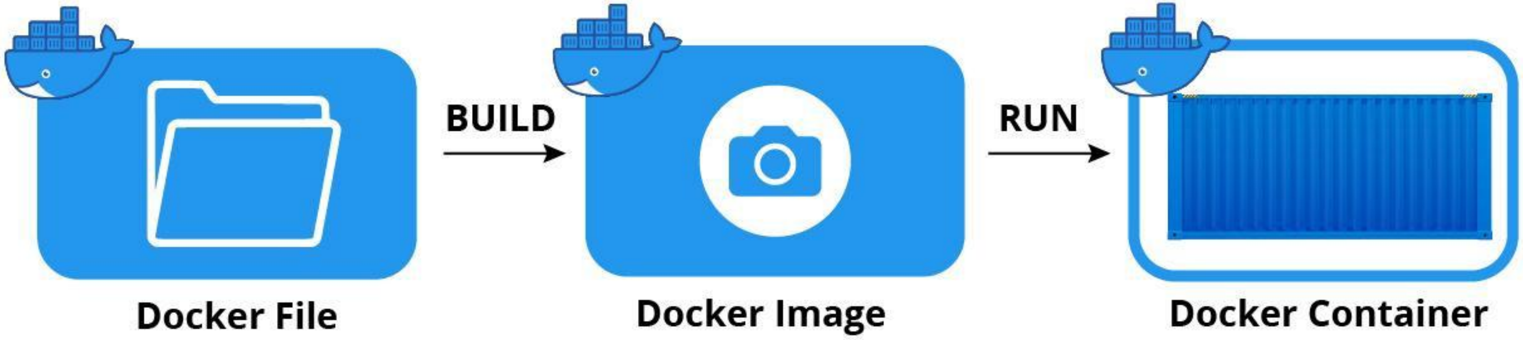


Docker File

What is the building process?



What is the building process?



Dockerfile

It is possible to build your own images reading instructions from a Dockerfile

```
FROM centos:7
RUN yum install -y python-devel python-virtualenv
RUN virtualenv /opt/indico/venv
RUN pip install indico
COPY entrypoint.sh /opt/indico/entrypoint.sh
EXPOSE 8000
ENTRYPOINT /opt/indico/entrypoint.sh
```

Dockerfile

It is possible to build your own images reading instructions from a Dockerfile

Base Image

```
FROM centos:7
RUN yum install -y python-devel python-virtualenv
RUN virtualenv /opt/indico/venv
RUN pip install indico
COPY entrypoint.sh /opt/indico/entrypoint.sh
EXPOSE 8000
ENTRYPOINT /opt/indico/entrypoint.sh
```

Dockerfile

It is possible to build your own images reading instructions from a Dockerfile

Install some tools

```
FROM centos:7
RUN yum install -y python-devel python-virtualenv
RUN virtualenv /opt/indico/venv
RUN pip install indico
COPY entrypoint.sh /opt/indico/entrypoint.sh
EXPOSE 8000
ENTRYPOINT /opt/indico/entrypoint.sh
```

Dockerfile

It is possible to build your own images reading instructions from a Dockerfile

Copy one file to container

```
FROM centos:7
RUN yum install -y python-devel python-virtualenv
RUN virtualenv /opt/indico/venv
RUN pip install indico
COPY entrypoint.sh /opt/indico/entrypoint.sh
EXPOSE 8000
ENTRYPOINT /opt/indico/entrypoint.sh
```

Dockerfile

It is possible to build your own images reading instructions from a Dockerfile

```
FROM centos:7
RUN yum install -y python-devel python-virtualenv
RUN virtualenv /opt/indico/venv
RUN pip install indico
COPY entrypoint.sh /opt/indico/entrypoint.sh
EXPOSE 8000
ENTRYPOINT /opt/indico/entrypoint.sh
```

Use port 8000 (in future)

docker run -p 8000:8000 your-image, Now, this will open port 8000 for us

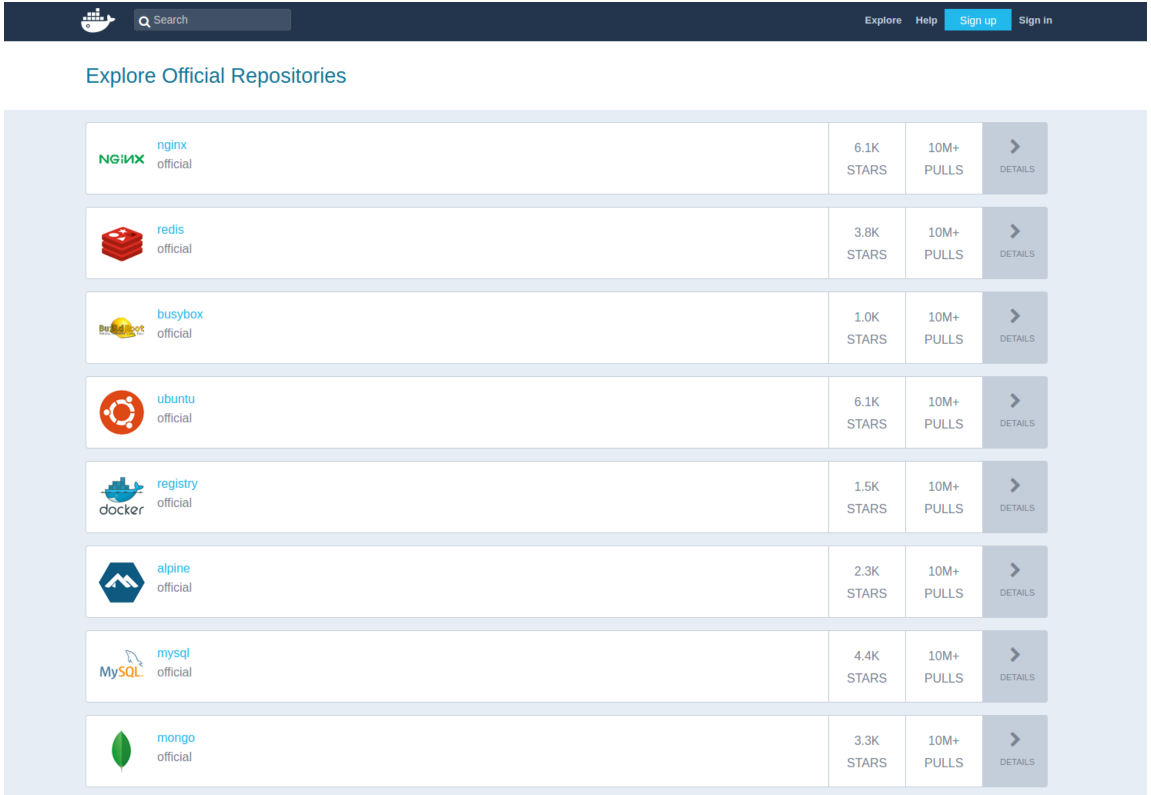
Dockerfile

It is possible to build your own images reading instructions from a Dockerfile

```
FROM centos:7
RUN yum install -y python-devel python-virtualenv
RUN virtualenv /opt/indico/venv
RUN pip install indico
COPY entrypoint.sh /opt/indico/entrypoint.sh
EXPOSE 8000
ENTRYPOINT /opt/indico/entrypoint.sh
```

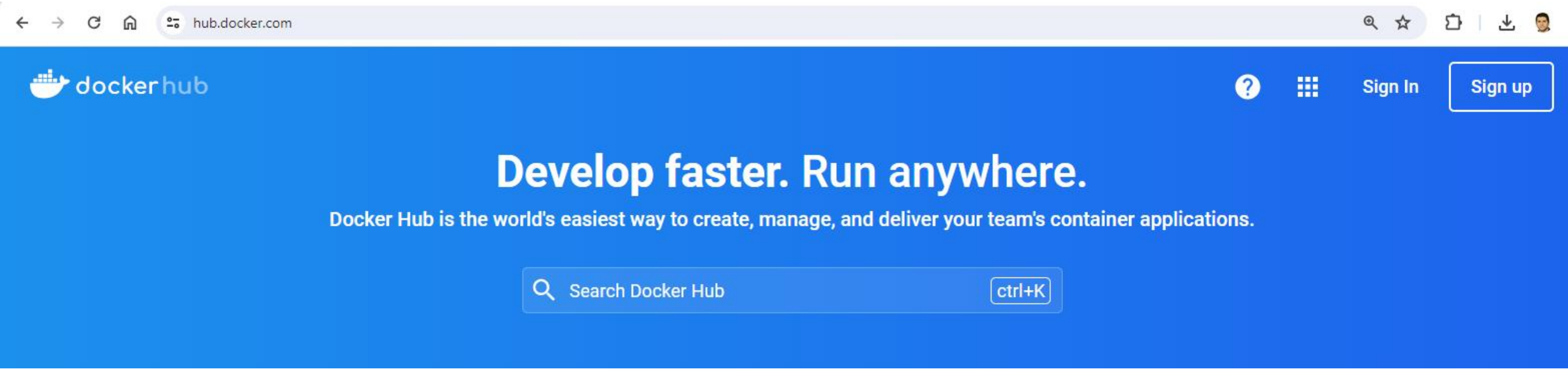
When container instantiated, this file will be run

Docker Hub


Search
Explore
Help
Sign up
Sign in
Explore Official Repositories
nginx
official
6.1K STARS
10M+ PULLS
DETAILS
redis
official
3.8K STARS
10M+ PULLS
DETAILS
busybox
official
1.0K STARS
10M+ PULLS
DETAILS
ubuntu
official
6.1K STARS
10M+ PULLS
DETAILS
registry
official
1.5K STARS
10M+ PULLS
DETAILS
alpine
official
2.3K STARS
10M+ PULLS
DETAILS
mysql
official
4.4K STARS
10M+ PULLS
DETAILS
mongo
official
3.3K STARS
10M+ PULLS
DETAILS

Docker Hub

First of all, create an account here



Docker compose

Allows to run multi-container Docker applications reading instructions from a docker-compose.yml file

```yaml
services:
  my-application:
    build: ./
    ports:
      - "8000:8000"
    environment:
      - CONFIG_FILE
  db:
    image: postgres
  redis:
    image: redis
    command: redis-server --save "" --appendonly no
    ports:
      - "6379"
```

Docker compose

Allows to run multi-container Docker applications reading instructions from a docker-compose.yml file

First Service

Second Service

```
services:
  my-application:
    build: ./
    ports:
      - "8000:8000"
    environment:
      - CONFIG_FILE
  db:
    image: postgres
  redis:
    image: redis
    command: redis-server --save "" --appendonly no
    ports:
      - "6379"
```

Some Docker commands

`docker run` - Use an image to run a container

Some Docker commands

`docker run` - Use an image to run a container

`docker pull` - Pull an image from a registry

Some Docker commands

`docker run` - Use an image to run a container

`docker pull` - Pull an image from a registry

`docker build` - Build a Docker image

Some Docker commands

`docker run` - Use an image to run a container

`docker pull` - Pull an image from a registry

`docker build` - Build a Docker image

`docker images` - List available Docker images

Some Docker commands

`docker run` - Use an image to run a container

`docker pull` - Pull an image from a registry

`docker build` - Build a Docker image

`docker images` - List available Docker images

`docker ps` - List running Docker containers

Some Docker commands

`docker run` - Use an image to run a container

`docker pull` - Pull an image from a registry

`docker build` - Build a Docker image

`docker images` - List available Docker images

`docker ps` - List running Docker containers

`docker exec` - Execute a command in a container

Some Docker commands

`docker run` - Use an image to run a container

`docker pull` - Pull an image from a registry

`docker build` - Build a Docker image

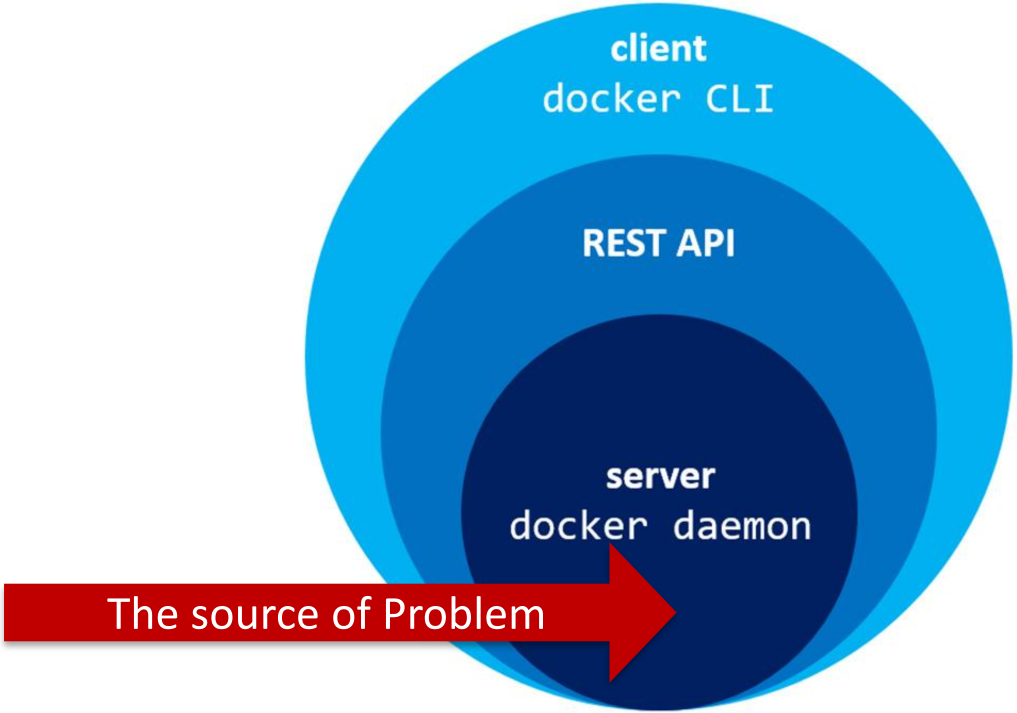
`docker images` - List available Docker images

`docker ps` - List running Docker containers

`docker exec` - Execute a command in a container

`docker stop` - Stop a running container

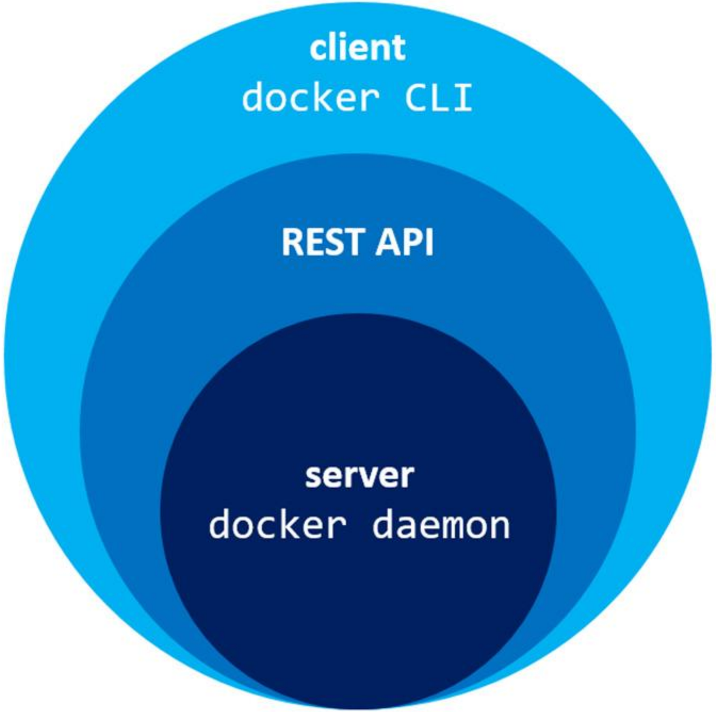
Docker is a flawless?



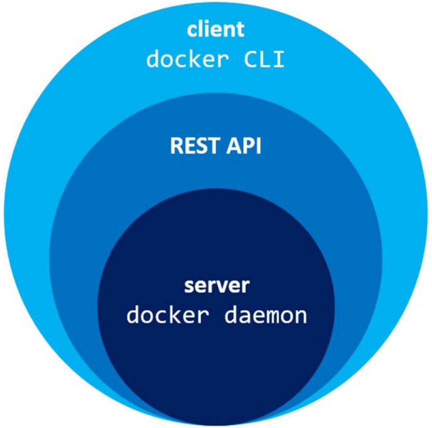
Docker Daemon

The root of the security issue is the Docker daemon (**dockerd**).

It runs with root privileges on your host machine. Any user or process that can talk to the Docker API can effectively gain root access to the entire host, making the API a critical security boundary.



Docker Daemon



This risk became a reality in 2026 with **CVE-2026-34040**.

CVE comes from *Common Vulnerabilities and Exposures*.

This vulnerability allows an attacker with API access to bypass authorization plugins (AuthZ) by sending a single, oversized API request. The authorization plugin would see an empty request and approve it, while the daemon processed the full, malicious request to create a privileged container with access to the entire host filesystem.

Any solution?

Docker Alternative

Kaniko is a tool to build container images from a Dockerfile, inside a container or Kubernetes cluster.

In early 2026, **CVE-2026-28406** was disclosed, a high-severity path traversal vulnerability in Kaniko that could allow an attacker to write files outside the build context and potentially achieve code execution.

This vulnerability was fixed in version 1.25.10 . This shows that while it removes one major class of risk (the daemon), it introduces its own set of potential vulnerabilities.

Keep Working with Docker BUT

- **Run Docker in Rootless Mode:**
- **Limit Access to the Docker API:**
- **Use an API Proxy/Firewall:**
- **Update and Stay Current:**

Thanks for watching!
Now, let's GOOOOOOOO for
Practical stuffs 🚀